

## > LAB 1

↳ 6 cells hidden

## > LAB 2

↳ 11 cells hidden

## > LAB 3

↳ 4 cells hidden

## > LAB 4

↳ 20 cells hidden

## > LAB 5

↳ 1 cell hidden

## > LAB 6

↳ 2 cells hidden

## > LAB 7

↳ 6 cells hidden

## > LAB 8

▶ ↳ 18 cells hidden

## ✓ LAB 9

**Hand written digit recognition using cnn**

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.models import Sequential
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0
# reshape for CNN
X_train_cnn = X_train.reshape(-1,28,28,1)
X_test_cnn = X_test.reshape(-1,28,28,1)

model = Sequential([

    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),

    Flatten(),

    Dense(128, activation='relu'),
    Dense(10, activation='softmax')

])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train_cnn, y_train,
          epochs=10,
          batch_size=64,
          validation_split=0.1)
```

```
test_loss, test_acc = model.evaluate(X_test_cnn, y_test)

print("Test accuracy:", test_acc)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-dataset/11490434/11490434> ————— 0s 0us/step  
 /usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base\_conv  
 super().\_\_init\_\_(activity\_regularizer=activity\_regularizer, \*\*kwargs)  
 Model: "sequential"

| Layer (type)                   | Output Shape       | Param # |
|--------------------------------|--------------------|---------|
| conv2d (Conv2D)                | (None, 26, 26, 32) | 320     |
| max_pooling2d (MaxPooling2D)   | (None, 13, 13, 32) | 0       |
| conv2d_1 (Conv2D)              | (None, 11, 11, 64) | 18,496  |
| max_pooling2d_1 (MaxPooling2D) | (None, 5, 5, 64)   | 0       |
| flatten (Flatten)              | (None, 1600)       | 0       |
| dense (Dense)                  | (None, 128)        | 204,928 |
| dense_1 (Dense)                | (None, 10)         | 1,290   |

Total params: 225,034 (879.04 KB)  
 Trainable params: 225,034 (879.04 KB)  
 Non-trainable params: 0 (0.00 B)

Epoch 1/10

844/844 ————— 63s 70ms/step - accuracy: 0.9470 - loss: 0.1765 - v

Epoch 2/10

844/844 ————— 70s 55ms/step - accuracy: 0.9841 - loss: 0.0511 - v

Epoch 3/10

844/844 ————— 47s 55ms/step - accuracy: 0.9895 - loss: 0.0349 - v

Epoch 4/10

844/844 ————— 47s 56ms/step - accuracy: 0.9917 - loss: 0.0260 - v

Epoch 5/10

844/844 ————— 46s 54ms/step - accuracy: 0.9936 - loss: 0.0196 - v

Epoch 6/10

844/844 ————— 46s 55ms/step - accuracy: 0.9949 - loss: 0.0155 - v

Epoch 7/10

844/844 ————— 46s 54ms/step - accuracy: 0.9959 - loss: 0.0122 - v

Epoch 8/10

844/844 ————— 82s 54ms/step - accuracy: 0.9964 - loss: 0.0106 - v

Epoch 9/10

844/844 ————— 82s 55ms/step - accuracy: 0.9975 - loss: 0.0077 - v

Epoch 10/10

844/844 ————— 46s 54ms/step - accuracy: 0.9972 - loss: 0.0086 - v

313/313 ————— 4s 13ms/step - accuracy: 0.9908 - loss: 0.0312

Test accuracy: 0.9908000230789185

```
import matplotlib.pyplot as plt
```

```

batch_sizes = [32, 64, 128]
histories = {}

for batch in batch_sizes:
    model = Sequential([
        Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
        MaxPooling2D((2,2)),

        Conv2D(64, (3,3), activation='relu'),
        MaxPooling2D((2,2)),

        Flatten(),
        Dense(128, activation='relu'),
        Dense(10, activation='softmax')
    ])

    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])

    print(f"\nTraining with batch size: {batch}")
    history = model.fit(X_train_cnn, y_train,
                        epochs=10,
                        batch_size=batch,
                        validation_split=0.1,
                        verbose=1)

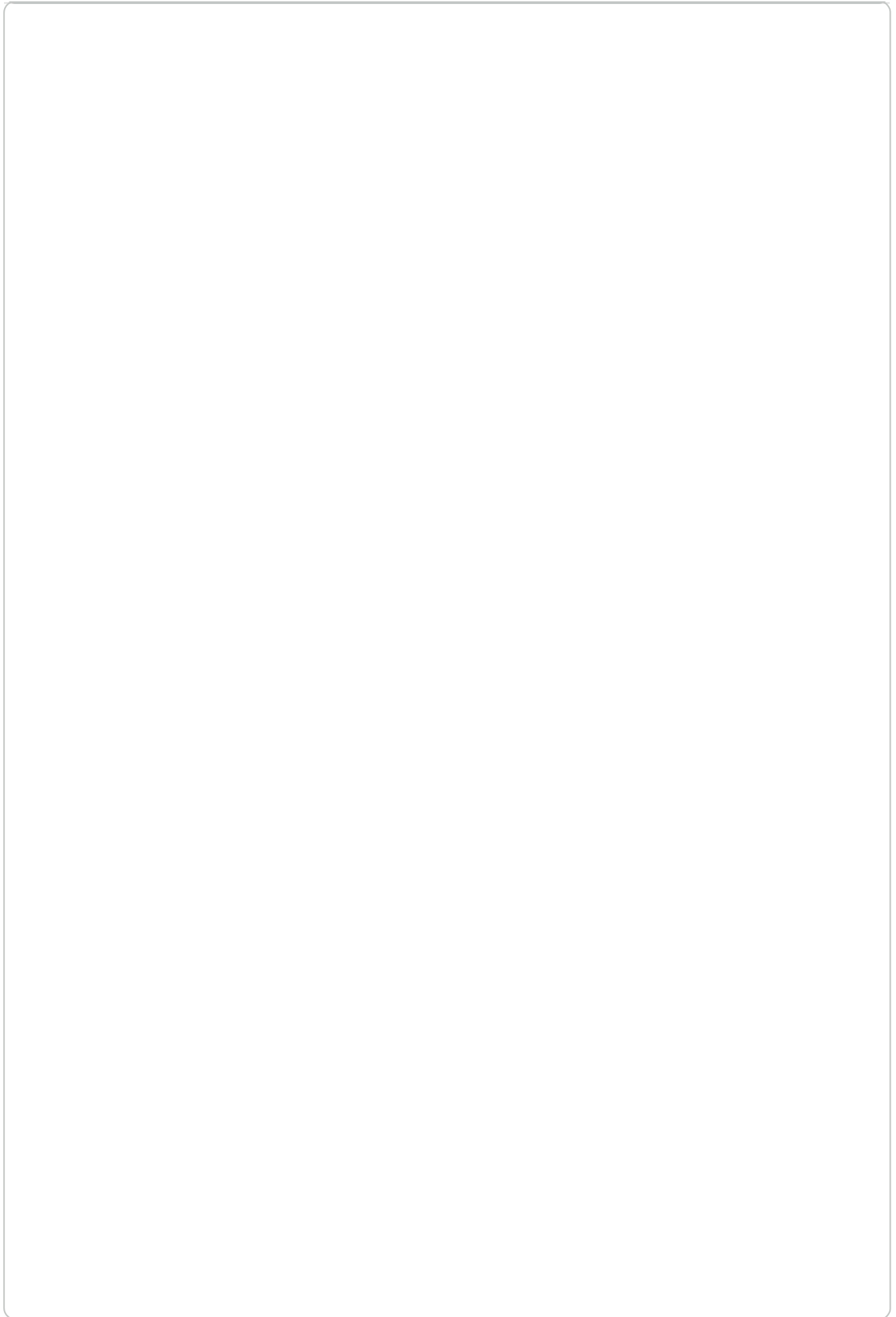
    histories[batch] = history

# Plot accuracy
plt.figure(figsize=(8,5))

for batch in batch_sizes:
    plt.plot(histories[batch].history['accuracy'], label=f'Train Acc (batch={batch})')
    plt.plot(histories[batch].history['val_accuracy'], linestyle='--', label=f'Val Acc (batch={batch})')

plt.title("Accuracy vs Epochs")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()

```



Training with batch size: 32

Epoch 1/10

**1688/1688**  57s 32ms/step - accuracy: 0.9579 - loss: 0.1391

Epoch 2/10

**1688/1688**  100s 43ms/step - accuracy: 0.9862 - loss: 0.043

Epoch 3/10

**1688/1688**  62s 31ms/step - accuracy: 0.9902 - loss: 0.0302

Epoch 4/10

**1688/1688**  55s 32ms/step - accuracy: 0.9927 - loss: 0.0228

Epoch 5/10

**1688/1688**  51s 30ms/step - accuracy: 0.9948 - loss: 0.0165

Epoch 6/10

**1688/1688**  53s 31ms/step - accuracy: 0.9952 - loss: 0.0136

Epoch 7/10

**1688/1688**  52s 31ms/step - accuracy: 0.9964 - loss: 0.0105

Epoch 8/10

**1688/1688**  53s 31ms/step - accuracy: 0.9973 - loss: 0.0086

Epoch 9/10

**1688/1688**  81s 30ms/step - accuracy: 0.9976 - loss: 0.0073

Epoch 10/10

**1688/1688**  84s 32ms/step - accuracy: 0.9978 - loss: 0.0060

Training with batch size: 64

Epoch 1/10

**844/844**  48s 55ms/step - accuracy: 0.9473 - loss: 0.1742 -

Epoch 2/10

**844/844**  46s 55ms/step - accuracy: 0.9838 - loss: 0.0515 -

Epoch 3/10

**844/844**  82s 55ms/step - accuracy: 0.9893 - loss: 0.0352 -

Epoch 4/10

**844/844**  82s 55ms/step - accuracy: 0.9914 - loss: 0.0273 -

Epoch 5/10

**844/844**  47s 55ms/step - accuracy: 0.9937 - loss: 0.0190 -

Epoch 6/10

**844/844**  82s 55ms/step - accuracy: 0.9949 - loss: 0.0154 -

Epoch 7/10

**844/844**  47s 56ms/step - accuracy: 0.9960 - loss: 0.0122 -

```
import matplotlib.pyplot as plt
```

```
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])
```

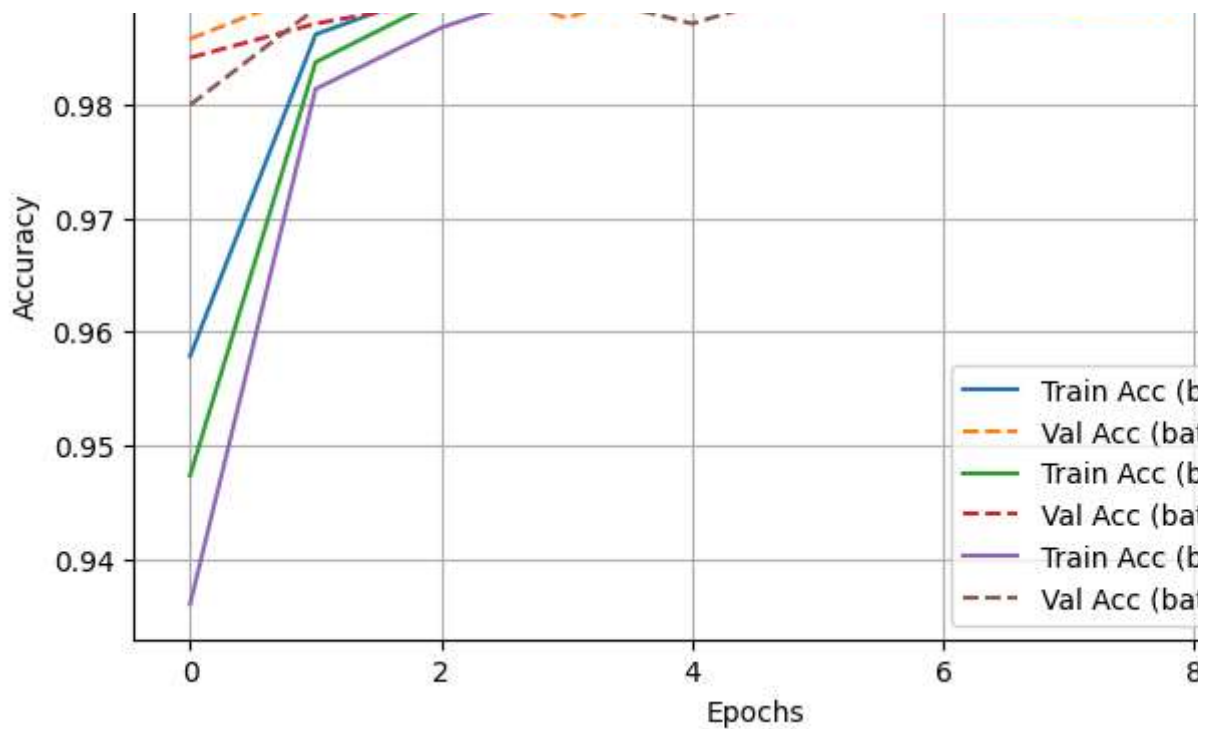
```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

```

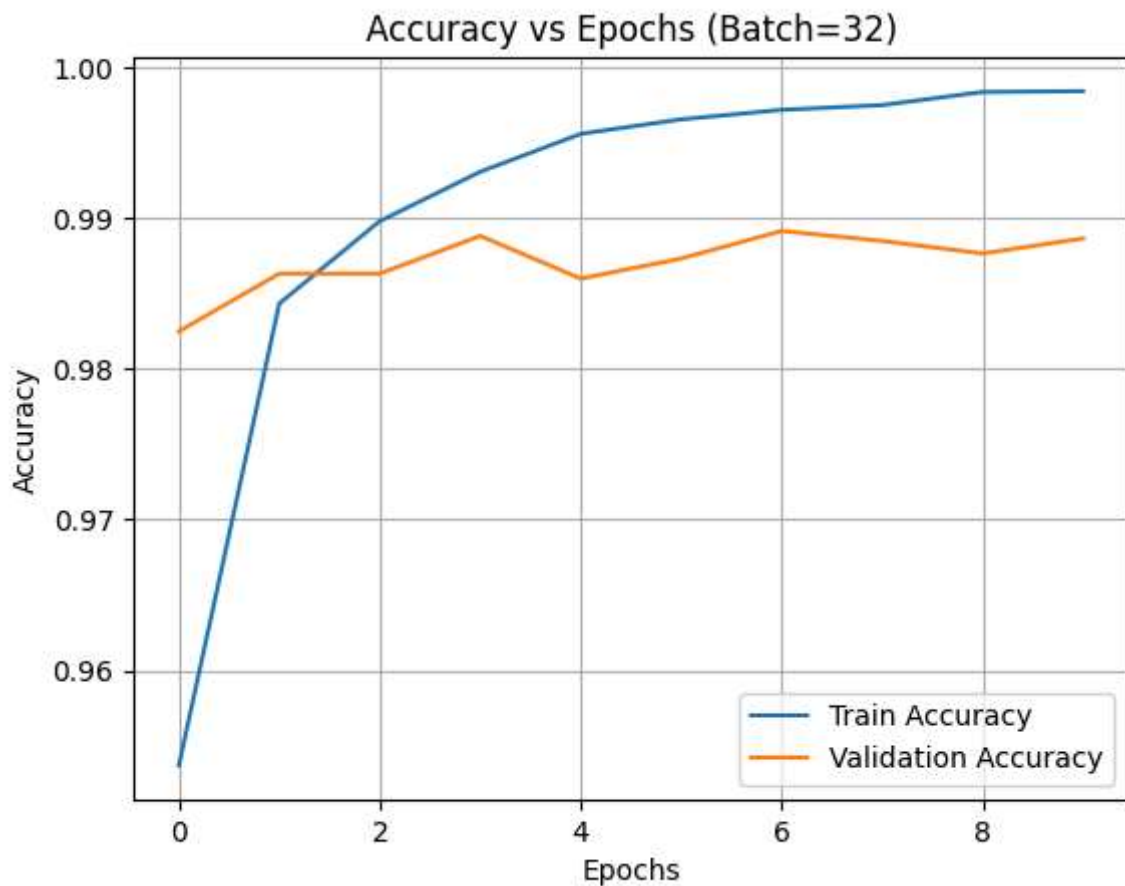
history = model.fit(X_train_cnn, y_train,
                    epochs=10,
                    batch_size=32,
                    validation_split=0.1)

# Plot accuracy
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Accuracy vs Epochs (Batch=32)")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()

```



Epoch 1/10  
**1688/1688** ————— 71s 39ms/step - accuracy: 0.9537 - loss: 0.1563 -  
 Epoch 2/10  
**1688/1688** ————— 39s 23ms/step - accuracy: 0.9844 - loss: 0.0514 -  
 Epoch 3/10  
**1688/1688** ————— 38s 23ms/step - accuracy: 0.9898 - loss: 0.0322 -  
 Epoch 4/10  
**1688/1688** ————— 42s 23ms/step - accuracy: 0.9931 - loss: 0.0214 -  
 Epoch 5/10  
**1688/1688** ————— 37s 22ms/step - accuracy: 0.9956 - loss: 0.0132 -  
 Epoch 6/10  
**1688/1688** ————— 40s 23ms/step - accuracy: 0.9966 - loss: 0.0097 -  
 Epoch 7/10  
**1688/1688** ————— 39s 23ms/step - accuracy: 0.9972 - loss: 0.0085 -  
 Epoch 8/10  
**1688/1688** ————— 37s 22ms/step - accuracy: 0.9975 - loss: 0.0072 -  
 Epoch 9/10  
**1688/1688** ————— 43s 23ms/step - accuracy: 0.9984 - loss: 0.0051 -  
 Epoch 10/10  
**1688/1688** ————— 37s 22ms/step - accuracy: 0.9984 - loss: 0.0047 -



```
import matplotlib.pyplot as plt
```

```
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),
```



```

        Flatten(),

        Dense(10, activation='softmax')
    ])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(X_train_cnn, y_train,
                   epochs=10,
                   batch_size=32,
                   validation_split=0.1)

# Plot accuracy
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Accuracy vs Epochs (Batch=32)")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid()
plt.show()

```

```

Epoch 1/10
1688/1688 ————— 42s 23ms/step - accuracy: 0.9319 - loss: 0.2368 -
Epoch 2/10
1688/1688 ————— 26s 15ms/step - accuracy: 0.9750 - loss: 0.0864 -
Epoch 3/10
1688/1688 ————— 25s 15ms/step - accuracy: 0.9808 - loss: 0.0647 -
Epoch 4/10
1688/1688 ————— 27s 16ms/step - accuracy: 0.9844 - loss: 0.0531 -
Epoch 5/10
1688/1688 ————— 27s 16ms/step - accuracy: 0.9865 - loss: 0.0449 -
Epoch 6/10
1688/1688 ————— 24s 14ms/step - accuracy: 0.9882 - loss: 0.0386 -
Epoch 7/10
1688/1688 ————— 26s 15ms/step - accuracy: 0.9901 - loss: 0.0333 -
Epoch 8/10
1688/1688 ————— 26s 16ms/step - accuracy: 0.9915 - loss: 0.0280 -
Epoch 9/10
1688/1688 ————— 27s 16ms/step - accuracy: 0.9925 - loss: 0.0240 -
Epoch 10/10
1688/1688 ————— 38s 14ms/step - accuracy: 0.9938 - loss: 0.0209 -

```